# 工具调用：让AI操作外部世界

- 来源: https://class.imooc.com/lesson/5283#mid=105032
- 阶段: 阶段四: 【框架应用】高效AI开发
- 周: 第8周 LangChain框架入门
- 模块: RAG集成：打造智能知识库
- 课节: 图文 1-2 工具调用：让AI操作外部世界

## 1-2 工具调用：让AI操作外部世界

## 工具调用：让AI操作外部世界

> [!TIP] 阅读指南 在前几节中，AI 只能"动嘴说"。从本节开始，我们将教它"动手做"。 你会学到如何用 @tool 给 AI 添加工具能力，并用 create_agent 让 AI 自动决定何时调用这些工具。 最后还会讨论 Agent 的使用边界——并非所有场景都需要 Agent，以及什么时候该用更强大的 LangGraph（第 7 章）。

## 从工具调用到 Agent

第 4 章中我们见过 Function Calling——AI 可以输出"调用某函数"的指令，但解析和执行的步骤需要手动完成。

Agent（智能体）

第一步：定义工具

用 @tool 装饰器把一个普通 Python 函数标记为 AI 可调用的工具：

```
from langchain_core.tools import tool

@tool
def get_weather(city: str) -> str:
    """查询指定城市的天气。"""
    weather_data = {"上海": "多云，20°C", "杭州": "晴，25°C"}
    return weather_data.get(city, f"暂无{city}的天气数据")
```

@tool 会自动分析函数名、参数类型和文档字符串，生成一份"说明书"发给 AI。AI 据此就知道 get_weather 是干什么的、需要什么参数。

第二步：创建 Agent

```
from langchain.agents import create_agent
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="qwen3.6-plus")
agent = create_agent(llm, [get_weather])
```

第三步：调用

```
result = agent.invoke({"messages": [("user", "上海天气怎么样？")]})
print(result["messages"][-1].content)

1
2
```

> [!TIP] 完整源码参考：samples/chapter8/tool_calling/01_weather_agent.py

Agent 的工作循环（推理→行动→观察→决策）在第 4 章已详细解释过，此处不再重复。需要理解的核心是：Agent 可以自主决定何时调用工具、用什么参数，并把结果整合成自然语言回答。

## 从单工具到多工具

当面对多个工具和逻辑依赖时，Agent 的价值更明显。例如提问"我明天去上海出差，穿什么合适？"——这个简单问题背后隐藏着两步逻辑：先查上海气温，再根据气温给出穿衣建议。

```
@tool
def get_weather(city: str) -> str:
    """查询城市天气。"""
    weather_data = {"上海": "多云，20°C"}
    return weather_data.get(city, f"暂无{city}天气数据")

@tool
def clothing_advice(temperature: int) -> str:
    """根据气温提供穿衣建议。"""
    return "穿短袖" if temperature > 18 else "穿外套"

# 创建 Agent 时传入多个工具
agent = create_agent(
    llm,
    [get_weather, clothing_advice],
    system_prompt="你是一个旅行助手。"
)

result = agent.invoke({"messages": [("user", "明天去上海出差，穿什么合适？")]})
```

Agent 内部的表现：先调用 get_weather 获取气温（20°C），再调用 clothing_advice(20) 得到"穿短袖"，最后综合回答。这就是多步推理——上一步的结果直接影响下一步的执行。

> [!TIP] 完整源码参考：samples/chapter8/multi_tools/01_multi_tools.py

## Agent 不是银弹

create_agent 虽然方便，但用多了你会发现 Agent 的思考循环（Loop）伴随成本：每次"推理→行动→观察"都是一次完整的 LLM 调用，token 消耗和延迟都会成倍增加。

工程上有一条简单原则：能用 Chain（线性流水线）解决的问题，就不要用 Agent。

| 场景 | Chain（线性） | Agent（智能体） |
| --- | --- | --- |
| 路径 | 预先定义，固定不变 | 动态规划，边走边看 |
| 适用 | 步骤固定的重复劳动 | 结果不确定的复杂推理 |
| 成本 | 低（Token 少，速度快） | 高（多次推理，有延迟） |

如果流程可以用第 2 节的 |（管道符）画出来，就不要用 Agent。只有当流程图里出现复杂的条件分支、需要 AI 动态判断时，Agent 才是合适的选择。

即便如此，create_agent 提供的只是一个"黑盒"的 Agent 循环。如果你想精确控制每一步的执行逻辑——比如某个步骤失败后自动重试、走降级策略、或等待人工审批——LangChain 内置的 Agent 就捉襟见肘了。这正是第 7 章 LangGraph 要解决的问题。

## ■ 学点英语

| 中文 | English | 音标 | 说明 |
| --- | --- | --- | --- |
| 工具装饰器 | @tool Decorator | /tuːl ˈdekəreɪtər/ | 将普通函数标记为 AI 可调用工具的装饰器 |
| 输入模式 | Input Schema | /ˈɪnpʊt ˈskiːmə/ | AI 描述工具参数结构和类型的说明书 |
| 多步推理 | Multi-step Reasoning | /ˈmʌlti step ˈriːzənɪŋ/ | 上一步结果直接影响下一步执行的推理链路 |
| 黑盒 | Black Box | /blæk bɑːks/ | 外部无法观察内部执行细节的封装体 |

## 下一节预告

掌握了 Agent 工具调用后，AI 已经能够"动手做事"。但要让 AI 访问海量的私有文档——游戏攻略、技术手册、产品文档——还需要集成 RAG。下一节会看到如何用 LangChain 的 RAG 组件快速构建知识库问答系统。